SL2DT-07: User Governance & Access

> **Series:** SL2DT — Sumo Logic to Dynatrace | **Notebook:** 7 of 11 | **Created:** April 2026 | **Last Updated:** 08/24/2026

Overview

Goal of this step: translate Sumo's RBAC model (roles + search filters + capabilities) into Dynatrace Platform IAM (groups + policies + bucket scoping). Do not defer this work — IAM decisions are made in Wave 1 of the migration, not at the end.

Governance drift — users without proper policies, buckets without proper scoping, SSO not mapped — causes cutover delays more often than query translation issues. Get this right before Wave 2.

Table of Contents

Prerequisites

Requirement	Details
Audience	Platform admins + security team
Inputs	`inventory/roles.json`, `users.json`, `rbac-mapping.md` from SL2DT-02
Dynatrace access — Platform Token	For querying runtime IAM state (`fetch dt.iam.policies`, audit events): `iam:groups:read`, `iam:policies:read`, `settings:objects:read`
Dynatrace access — OAuth client	**The Terraform resources used in this notebook's HCL examples (`dynatrace_iam_group`, `dynatrace_iam_policy`, `dynatrace_iam_policy_bindings_v2`) are OAuth-client-only** — a Platform Token cannot drive them. Provision an OAuth client (`DT_CLIENT_ID` / `DT_CLIENT_SECRET` / `DT_ACCOUNT_ID`) with scopes `account-idm-read`, `account-idm-write`, `iam-policies-management`, `account-env-read`
SSO context	Existing IdP details (Okta, Entra, Ping)
Prior reading	IAM-01 (Platform IAM fundamentals), IAM-07 (bucket policies)

1. What You'll Produce

Artifact	Purpose
`iam/groups.tf`	Terraform for IAM groups (one per Sumo role)
`iam/policies.tf`	Policies (scoped by bucket)
`iam/bindings.tf`	Group→policy bindings
`iam/sso-mapping.md`	IdP claim → group membership rule
`iam-rollout-plan.md`	Sequence: shadow-mode → cut-over → retire-legacy

2. Sumo → Dynatrace IAM Mapping Model

The Three Layers

```
Sumo Role              Dynatrace IAM Model
═══════════════        ═══════════════════════
Role name       →      Group name
Capabilities    →      Policy permissions
Search filter   →      Policy bucket/scope filter
User assignment →      Group membership (often via SSO claim)
```

Example Mapping

Sumo role: `prod-readers`
- Search filter: `_sourceCategory=prod/*`
- Capabilities: Dashboard view, Search, Monitor view (no edit)
- Assigned users: 82

Dynatrace equivalent:

```
# Group
resource "dynatrace_iam_group" "g_prod_readers" {
  name = "g_prod_readers"
  description = "Read-only access to production logs + dashboards"
}

# Policy (bucket-scoped)
resource "dynatrace_iam_policy" "p_prod_readers" {
  name = "p_prod_readers"
  environment = var.tenant_uuid
  statement_query = &lt;&lt;-EOT
    ALLOW storage:logs:read, storage:events:read, storage:metrics:read
    WHERE storage:bucket-name = "custom_logs_prod";
    ALLOW document:documents:read;
  EOT
}

# Binding
resource "dynatrace_iam_policy_bindings_v2" "b_prod_readers" {
  group = dynatrace_iam_group.g_prod_readers.id
  environment = var.tenant_uuid
  policy = [dynatrace_iam_policy.p_prod_readers.id]
}
```

> **Critical — `dynatrace_iam_policy_bindings_v2` re-assigns all policies bound to a group on every apply.** Per the `dynatrace_iam_policy_bindings_v2` resource docs (Dynatrace provider docs) — also mirrored, JavaScript-free, at the provider repo (Dynatrace GitHub): *"This resource re-assigns all policies bound to a group, so every policy that should remain bound must be specified in the configuration; otherwise, it will be unbound."* The single-policy example above is illustrative — if `g_prod_readers` already has other policies bound (in Dynatrace itself or in a separate Terraform resource), applying this block with only `p_prod_readers` listed will **unbind those other policies**. List every policy that should remain bound to the group in the same `policy = [...]` array.

Policy Language

Dynatrace policies use a declarative grant syntax:

```
ALLOW &lt;permission&gt;, &lt;permission&gt;, ...
  WHERE &lt;condition&gt;;
```

Common bucket-scope conditions:

- `storage:bucket-name = "bucket_x"`
- `storage:bucket-name IN ("a", "b", "c")`
- `storage:table = "logs"`
- `document:owner-id = "$subject"` (user owns the document)

3. Bucket-Scoped Policies — Implementation

Bucket scoping is the primary isolation mechanism. Apply it for every team/environment boundary.

Policy Patterns

Pattern 1 — Read-only to one bucket

```
ALLOW storage:logs:read, storage:events:read
  WHERE storage:bucket-name = "custom_logs_prod";
```

Pattern 2 — Read all, write via a deliberate unscoped grant

```
ALLOW storage:logs:read
  WHERE storage:table = "logs";
// Storage writes cannot be condition-scoped (live-verified 07/2026) —
// grant unscoped, only to the team's ingest service user, and control
// where data lands via OpenPipeline routing, not IAM conditions.
ALLOW storage:logs:write;
```

Pattern 3 — Admin on multiple buckets

```
ALLOW storage:logs:read, storage:events:read
  WHERE storage:bucket-name LIKE "custom_logs_prod%";
ALLOW storage:logs:write;
ALLOW document:documents:read, document:documents:write;
ALLOW settings:objects:read, settings:objects:write
  WHERE settings:schema-id = "builtin:monitoring.slo";
```

> **Storage writes cannot be bucket-scoped (live-verified 07/2026):** `storage:logs:write WHERE storage:bucket-name ...` is rejected with `Invalid condition name`. Grant writes unscoped in a deliberately-assigned policy and route data via OpenPipeline. Wildcard permissions (`storage:*:read`, `ALLOW *`) are also rejected — enumerate.

Pattern 4 — Full admin (limit to platform engineering)

Do not author a wildcard policy (`ALLOW *` is rejected by the API). Bind the built-in **Admin User** managed policy — or, for a classic-parity custom grant, combine `environment:roles:viewer` + `environment:roles:manage-settings` with enumerated storage/settings permissions.

Avoid

- Binding admin-level policies to team-level groups. Escalation risk.
- Broad enumerated grants where a scoped read would do — harder to audit.
- Read permissions without WHERE clauses — silently grants tenant-wide access. (Storage *writes* cannot take WHERE at all — grant them deliberately and sparingly.)

Validate Policies

After applying, verify a test user in the group can (and cannot) do what the policy says:

> **For teams spanning multiple source categories by component type** (e.g., a database team needing access to `_sourceCategory=*/db/*` regardless of application), a structured `comp:<component>/bu:<business-unit>/app:<application>` security context format enables a single `MATCH('comp:db*')` boundary to work across all applications. See **IAM-04: Policy Authoring** and **ORGNZ-06: Security Context** for the design pattern.

```
// Verify recent IAM / policy activity from the audit log
//
// Corrected 08/12/2026 — the old cell had two defects. `dt.iam.policies` is
NOT a Grail data object
// at all (UNKNOWN_DATA_OBJECT): IAM policies live in Account Management and
are reachable only via
// the Account Management API, not DQL. And `filter name startsWith "p_prod"`
used an infix form that
// does not exist — startsWith is a FUNCTION, startsWith(name, "p_prod").
//
// What IS queryable is the audit trail of IAM activity, as AUDIT_EVENT
records:
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| summarize calls = count(), by:{event.type, user.email}
| sort calls desc
| limit 20

// For the policy definitions themselves, use the Account Management API — the
IAM series covers
// enumerating policies, bindings and groups programmatically.
```

4. SSO & User Provisioning

SCIM / SAML Integration

Most migrations preserve the same IdP (Okta / Entra / Ping). Map IdP groups to Dynatrace groups via SAML assertion attribute or SCIM group push.

Okta example — SAML assertion:

```
dynatrace.group = "g_prod_readers,g_payments_team"
```

Dynatrace parses the `dynatrace.group` attribute on login and assigns memberships.

SCIM push (preferred for lifecycle management):
- Okta SCIM connector writes group membership on user provisioning
- Membership changes propagate automatically
- Deprovisioning is immediate

User Lifecycle

Event	Sumo Behavior	Dynatrace Behavior
New hire	Manual add in Sumo admin	SCIM push → auto-assigned to groups
Team change	Role manually reassigned	SCIM push reassigns group memberships
Termination	Manual disable	SCIM push removes all memberships

Provisioning Tracker

```
| User | Sumo Role | Target DT Group | SSO Push Confirmed |
|------|-----------|-----------------|--------------------|
| jane@example.com | prod-readers | g_prod_readers | ✅ 2026-05-01 |
| ... |
```

5. Role-Based Dashboard/Monitor Access

Dashboards, notebooks, and workflows have their own access model (via `document:documents:*` permissions) separate from bucket scoping.

Document Permission Levels

Permission	Effect
`document:documents:read`	Can view own + shared
`document:documents:write`	Can edit own + shared
`document:documents:read WHERE document:owner-id = "$subject"`	Own only
`document:documents:read WHERE document:shared-with = "$subject"`	Shared with me
`document:documents:admin`	Can modify any document (platform admins only)

Default Shares

Set dashboard defaults during migration so new dashboards are readable by the team:

- Every team dashboard: shared with the team's IAM group (read)
- Every team notebook (operational): shared with team (read)
- Every team workflow: owned by team admin group (read/write)

Content Folders

Dynatrace Documents have a flat namespace with tags/shares, not folder hierarchy like Sumo. Map Sumo folders to tags:

Sumo Folder	Dynatrace Tag
`Payments/Prod`	`team:payments`, `env:prod`
`Identity/Shared`	`team:identity`
`Archive`	`status:archived`

6. Audit Trail Parity

Sumo's audit log captures every user action. Dynatrace has equivalent via `audit` events in Grail.

Sumo audit categories → Dynatrace equivalents

Sumo	Dynatrace
Login / Logout	`event.category = "authentication"`
DashboardViewed / DashboardEdited	`event.category = "document"` with action
MonitorCreated / MonitorUpdated	`event.category = "settings"`
Search executed	`event.category = "storage.query"`
User/Role changes	`event.category = "iam"`

Query audit events

```
// Dynatrace audit events
fetch events, from:-24h
| filter event.category == "iam"
| summarize c = count(), by:{event.action, user.name}
| sort c desc
```

Retention

Audit data goes in the `audit_logs` bucket (from SL2DT-03). Retention must match compliance requirements (usually 365d+). Verify in the bucket config:

```
resource "dynatrace_platform_bucket" "audit_logs" {
  name      = "audit_logs"
  retention = 365
}
```

7. Governance Pitfalls to Avoid

Pitfall 1 — "We'll do IAM at the end"

Deferring IAM means rebuilding ingest (bucket decisions are coupled to policy scope). Do IAM in Wave 1.

Pitfall 2 — Overly broad initial groups

Tempting: start with one group `all-users` with `ALLOW *`. Migration pressure compounds: "we'll tighten later." Later never comes, and when it does, revoking access breaks people's workflows.

Fix: start with the Sumo role structure as-is (one DT group per Sumo role, same scope). Refactor later with data.

Pitfall 3 — Unscoped dashboard writes

A dashboard created by a team member ends up without share settings, then becomes unreachable when they leave. Every dashboard should have a group owner, not a user owner.

Fix: enforce via PR review during SL2DT-06. Check the default-share policy.

Pitfall 4 — SSO group-name mismatch

Okta pushes group name `payments-platform-prod`; Dynatrace group is `g_prod_readers`. Users log in but get no permissions.

Fix: document SSO-claim → DT-group mapping in `iam/sso-mapping.md`. Test with at least one user per group before cutover.

Pitfall 5 — Missing audit events for bucket-level access

Bucket read/write permissions are evaluated per-query. Audit events must include `storage:bucket-name` to trace who queried what.

Fix: verify audit retention + query patterns in the audit bucket. See OPLOGS-09 for audit-query patterns.

8. Step Exit Criteria

G7 — Governance Ready

- [ ] IAM groups created (one per Sumo role)
- [ ] Policies created with bucket scoping
- [ ] Group↔policy bindings applied
- [ ] SSO mapping documented and tested with at least one user per group
- [ ] Dashboard/notebook share defaults configured
- [ ] Audit events flowing into `audit_logs` bucket with ≥365d retention
- [ ] Rollout plan documented (shadow-mode → cutover → retire legacy)

Next step: SL2DT-08 — Automation & GitOps (Monaco/Terraform for config promotion, CI/CD integration).

---

11. References

Dynatrace IAM

- Identity and access management (DT docs)
- Manage user permissions and policies (DT docs)
- Access tokens (DT docs)
- Platform tokens (DT docs)

Sumo Logic users and roles (source)

- Sumo Logic users and roles (Sumo Logic docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace or Sumo Logic. Always verify information against the official Dynatrace documentation and Sumo Logic documentation.